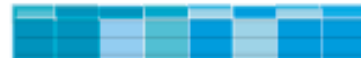




UTN

Universidad Tecnológica Nacional
Ingeniería en Sistemas de Información

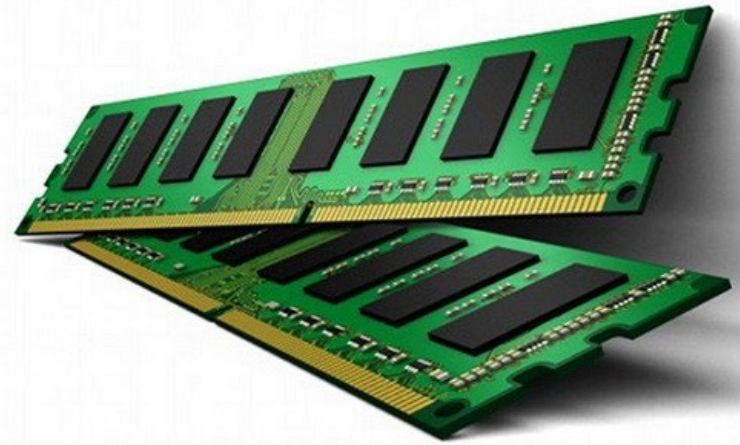


ARCHIVOS

Parte I

Ing. María Inés Seguenzia

Memoria principal RAM



Es donde reside el procesamiento de datos actual. Es la memoria donde se almacenan **temporalmente** los datos que se están procesando o se van a procesar. Es volátil, pierde su contenido cuando se apaga el equipo.



Memoria secundaria, auxiliar, periférica o externa



Es el conjunto de dispositivos y soportes de almacenamiento de datos que conforman el subsistema de memoria de la computadora, donde los datos se almacenan por un largo tiempo o **permanentemente**.

Qué es un ARCHIVO?

Un **archivo** es una estructura de datos que permite manejar grandes volúmenes de datos y conservarlos después de la finalización del programa ya que no se guardan en la memoria principal, sino en la secundaria. (**PERSISTENCIA de Datos**). Además, una vez que fue creado y grabado en un dispositivo externo puede ser utilizado por cualquier otro programa ya sea para obtener datos del archivo y/o para modificar los datos del mismo siempre y cuando la estructura interna del archivo sea conocida por el programador.

Qué elementos podremos guardar en un archivo?

registros, arreglos, caracteres, reales, enteros, etc...

Así tendremos, **archivos de registros**.

Es decir, los elementos del archivo pueden ser de tipo Simples o Estructurados.

**Ojo!!! Tampoco
podemos
mezclar tipos de
datos dentro de
un archivo!!!**

Es una estructura Dinámica

No es necesario definir previamente la longitud máxima de la estructura, dado que el límite de información a almacenar está dada por la capacidad física del medio de almacenamiento.



Clasificación de Archivos según su acceso y forma de organización

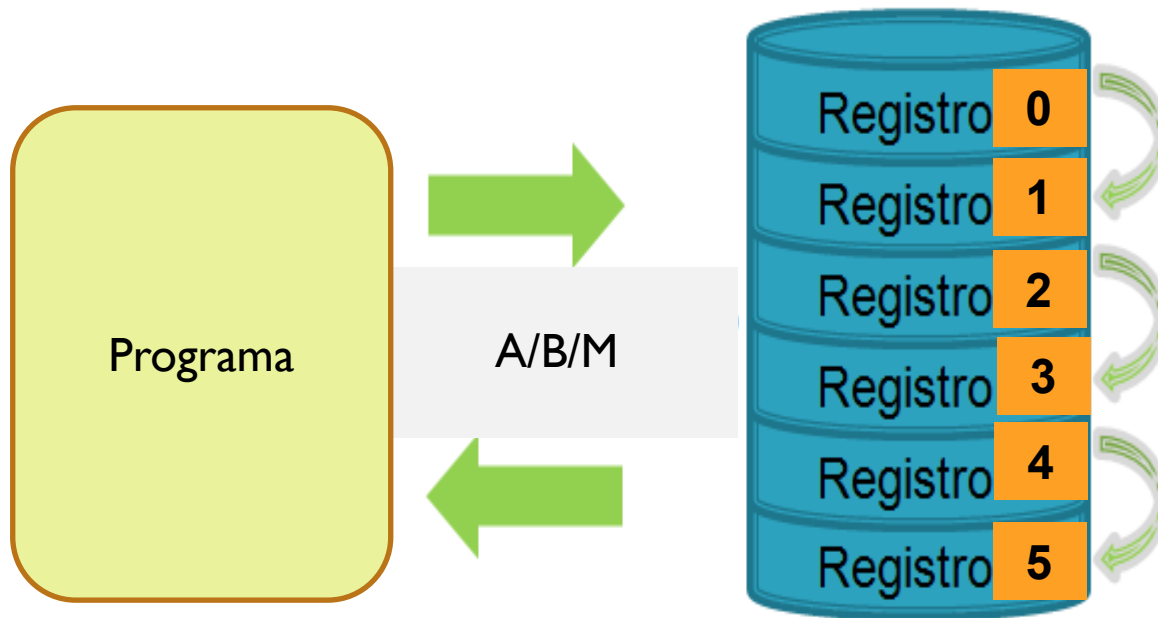
Secuenciales

Directos

Indexados

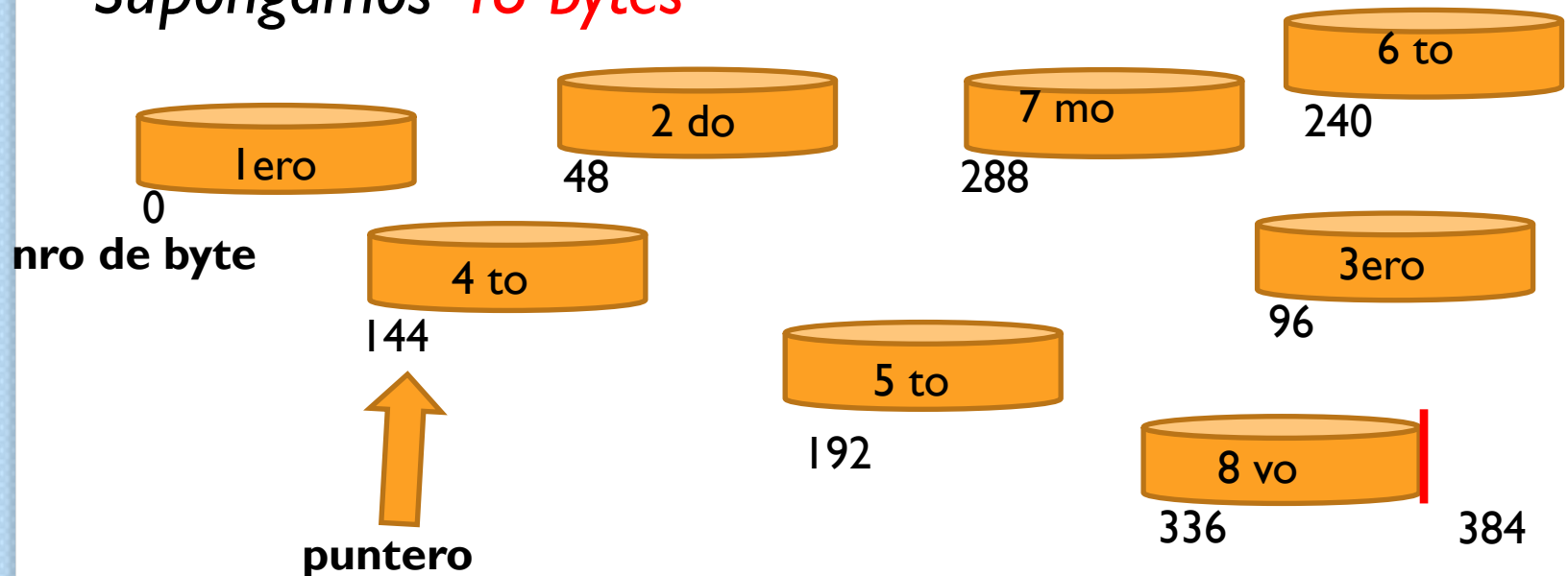
**En este curso
trabajaremos con
archivos de acceso
Directo!**

- **Secuenciales:** para acceder a un registro, se debe pasar por los anteriores, no se puede acceder directamente.



- de **Acceso Directo**: se puede acceder a cualquier registro directamente, sin necesidad de pasar por los anteriores. Para esto, cada registro tendrá a modo de índice un número que lo identifica (byte), correlativo desde el cero en adelante que le permita al programa, mediante algún cálculo, ir directamente a la posición donde está almacenado el registro. Los registros son de longitud fija.

Supongamos **48 bytes**





**Debemos leer o
grabar en el
archivo de a UN
REGISTRO POR
VEZ!!**

legajo	Nombre	Comisión	Carrera	N	Prome
--------	--------	----------	---------	---	-------

nro de byte
de 0 en adelante

**Puntero-
File Pointer**



Ram o memoria
principal

Bus de Datos



Memoria Auxiliar

Algunas consideraciones...

- En informática, un **búfer** (del inglés, **buffer**) es un espacio de memoria en el que se almacenan datos de manera temporal.
- Como no se puede tener todo el archivo completo en memoria principal, deberemos ir trayendo al buffer los registros uno a uno... a esta acción se le dice “*leer del archivo*”.
- No debemos confundir la organización del archivo con los recorridos ó tipos de búsqueda ya que nosotros trabajaremos con archivos Directos y utilizaremos sobre ellos búsqueda secuencial, dicotómica y podremos pararnos también en posición relativa tal cual como lo hemos venido haciendo con estructuras de tipo arreglos.

Algunas consideraciones más...

- Los datos que se guardan en un archivo se representan en sistema binario y por cada dato (campo) se utilizan tantos bytes como sea necesario para representarlo. Por lo tanto, el tamaño total del archivo es la cantidad total o sumatoria de bytes que contiene por cada dato.
- Si bien todos los archivos representan su contenido en base al sistema binario, en la práctica es común hacer una distinción entre **Archivos de Texto** y **Archivos Binarios**

Archivos de texto

Contienen datos almacenados como una

- Los bits de los archivos de texto representan caracteres.
- Contienen solamente datos de texto
- Todos los bytes del archivo son interpretados como caracteres (en base a la tabla ASCII). Se asume que el desarrollador quien organiza todos esos bytes representa caracteres que pueden ser visualizados en pantalla. La excepción del salto de línea en python es `/n`. Desde ahora en adelante todos tienen una representación visual.
- Los archivos de código fuente que realizamos en python son archivos de texto que pueden abrirse con el IDLE o con cualquier editor de texto.



Todos pesan lo mismo en bytes

Archivos binarios

serie de bits (valores binarios de 1s y 0s)

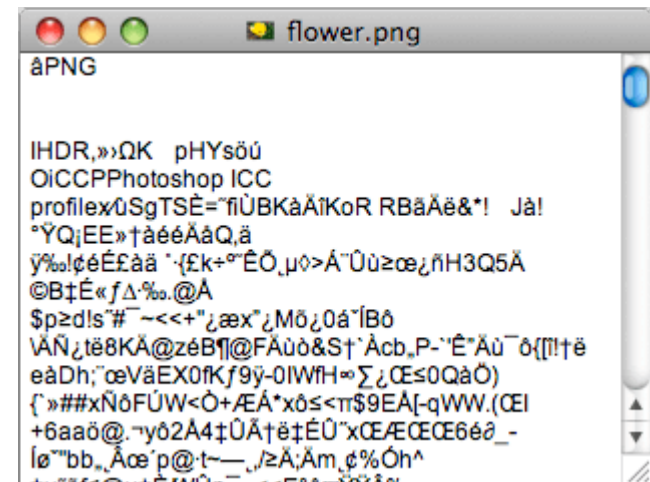
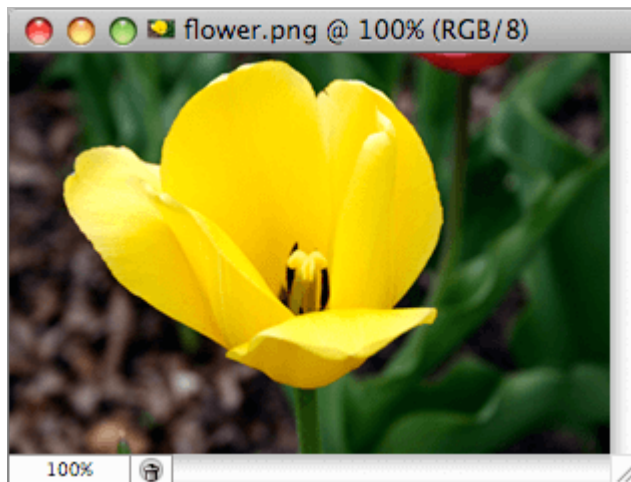
- Los bits en los archivos binarios representan datos personalizados.
 - Pueden contener tanto datos binarios personalizados como también de texto.
- El desarrollador quien organiza los datos en un formato que contiene la información necesaria para la aplicación a desarrollar. Los archivos binarios pueden guardar múltiples tipos de datos en el mismo archivo, tales como datos de audio, imágenes y video, enteros, booleanos, cadenas, registros. Estos datos pueden ser interpretados por los programas de apoyo, pero se mostrarán como texto ilegible en un editor de texto.

4byte	16byte	2	8bye
-------	--------	---	------

Cada campo tiene distinto peso

**Nosotros
usaremos
archivos binarios
en los que se
guardarán
Registros de
longitud fija.**

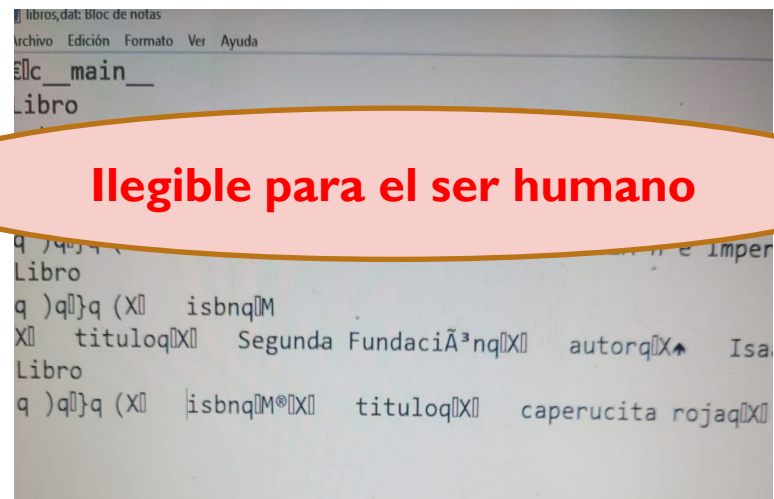
Ejemplo de un archivo de imagen .Png abierto en un visor de imágenes y en un editor de texto.



El visor de imágenes reconoce los datos binarios y muestra la imagen. Cuando la imagen se abre en un editor de texto, los datos binarios se convierten en texto irreconocible.

Ejemplo de un archivo de registros.dat abierto en un editor de texto.

odcli	nombre	dirección	localidad	teléfono
5	Páez Alicia	Av. Suárez 188	Capital	4856 7835
9	Rodríguez José	Av. Libertador 2820	Capital	4356 5568
10	Pérez Matilde	Rivadavia 8868	Mantilla	
14	Borg Cristian	Av. Libertador 2820	Capital	4331 6051
16	Pérez José	Pueyrredón 344	Palomar	4682 2331
19	Fiori Amelia	Castro 2253	Palomar	4697 8879
22	Pérez Guillermo	Pueyrredón 994	La Luisa	01 3 23238
29	Cassini Mirta	Ramón L. Falcón 1827	Palomar	4343 7856
31	López Ramiro	Nicaragua 388	Tablada	
33	Miguez Georgina	Hipólito Irigoyen 6838	San Martín	
36	Gómez Alvaro	Av. Asamblea 909	Capital	4484 7594
40	Pérez Emilio	Tucumán 338	Palomar	4332 9758
49	Valdéz Emérito	Arévalo 443	Palomar	4445 4587
50	Cuccio Hugo	Somellera 145	La Luisa	01 9 22399
72	Gómez Francisco	Rivadavia 228	La Manuelita	014990934



```
libros.dat: Bloc de notas
Archivo Edición Formato Ver Ayuda
c_main_
Libro
q )q }q (X isbnqIM
X tituloqX Segunda FundaciÃ³nqX autorqX Isa
Libro
q )q }q (X isbnqIM tituloqX caperucita rojaqX
```

Ilegible para el ser humano

Operaciones básicas sobre Archivos Binarios en Python

Un archivo es un objeto que será guardado físicamente en un lugar con un nombre.

“c:\\ayed\\alumnos.dat” #Path o ruta de acceso o dirección de memoria Física

Podemos asignarla a una variable. Por ejemplo:

ArcFisiAlu= “c:\\ayed\\alumnos.dat”

El mismo se crea y se deja disponible para el programador a través de la función interna **OPEN()** que contiene 2 parámetros:

- el 1ero es el nombre del archivo que puede contener su ruta,
- el 2do es el modo de apertura del archivo

ArcLogAlu = OPEN (ArcFisiAlu, "w+b")

variable o nombre lógico de tipo archivo.

W+ crea o reemplaza físicamente el archivo, b es para indicar archivo binario

Modo grabación y lectura. Si existe lo borra y lo crea nuevamente

Distintos modos de apertura de un Archivo Binario en Python

rb	Se abre para solo lectura. No se crea si no existe. El puntero se ubica al comienzo del archivo.
wb	Se abre para solo grabación. Si ya existe, se elimina su contenido. Si no existe se crea El puntero se ubica al comienzo del archivo.
ab	Se abre para solo append. Todas las grabaciones se hacen al final del archivo. Si no existe se crea uno nuevo.
r+b	Se abre para lectura y grabación, pero debe existir previamente. Si no existe no se crea. El puntero se ubica al comienzo del archivo.
w+b	Se abre para lectura y grabación. Si ya existe, se elimina su contenido. Si no existe se crea El puntero se ubica al comienzo del archivo.
a+b	Se abre para lectura y append. Todas las grabaciones se hacen al final del archivo. Si ya existe preserva su contenido. Si no existe lo crea.

Cerrar un archivo

Desvincula el archivo físico de su nombre o variable lógica. Para cerrar un archivo abierto utilizamos...

Variable-Lógica- tipo-archivo.close()

ArcLogAlu.close()

Luego de cerrar la variable lógica queda indefinida.

Leer del archivo

RECORDAR!!!
Leer y grabar AVANZAN
EL PUNTERO
AUTOMÁTICAMENTE!!!

Significa traer (levantar) el registro (o elemento) del archivo a memoria principal para trabajarlo...

Cuál leo? Cuál traigo?

Aquel que se encuentre donde está posicionado el puntero en ese momento.

Grabar en el Archivo

Cuando agregamos elementos o registros al archivo ó queremos modificar(actualizar) algún campo debemos efectivizar dicha modificación y grabarlo...

Dónde se graba? *Donde esté posicionado el puntero.*

Siempre se Lee o se graba el
registro COMPLETO!!!

Lectura y Grabación Serealización



La **serealización** es un proceso por el cual el contenido de una variable normalmente de estructura compleja (registro) se convierte automáticamente en una secuencia de bytes listos para ser almacenados en un archivo, pero de forma tal que luego esa secuencia puede recuperarse desde el archivo y volver a crear con ella el registro original.

Para ello en Python deberemos importar la siguiente biblioteca:

Import pickle # importa módulo **pickle** para serealización de archivos. Este módulo nos provee las funciones **LOAD()** y **DUMP()** para leer y grabar del archivo.

Si bien se utiliza para variables complejas como registros también puede usarse para manipular variables simples.

Supongamos este ejemplo...

```
class Alumno:  
    def __init__(self): # constructor dentro de la clase Alumno  
        self.legajo = 0  
        self.comision= 0  
        self.nombre = " "  
        self.carrera = " "  
        self.notas = [0]*3  
        self.promedio= 0.00
```

Programa Principal

debo declarar variables lógicas de tipo archivo y registro

ArcFisiAlu= "c:\\ayed\\alumnos.dat"

ArcLogAlu = open (**ArcFisiAlu**, "w+b")

RegAlu = **Alumno**() # necesitamos tener una variable lógica de tipo un registro para moverme en el bus de datos

Load() es un método de Pickle para leer un registro... traerlo a memoria

Variable-lógica-tipo-registro = pickle.load (arch-lógico)

RegAlu = pickle.load(arcLogAlu)

Así, traigo el registro **COMPLETO** a memoria !!!

Cuál trae??? ... Donde esté parado el puntero.

Dump() es un método de Pickle para grabar un registro al archivo

pickle.dump(variable-lógica-tipo-registro, arch-lógico)

pickle.dump(RegAlu, arcLogAlu)

arcLogAlu.flush()

Dónde graba?? ... Donde esté parado el puntero.

Flush() se utiliza para Forzar la grabación de dump() en el archivo.

Resumen de bibliotecas que deberemos importar



import os # permite realizar operaciones dependientes del OS =
Sistema Operativo

`os.makedirs('ayed')` # crea una carpeta

`os.getcwd` # obtener la ruta de trabajo en un string

`os.path.getsize (arch-fisico)` # muestra el tamaño del archivo en bytes.

Si es igual a cero entonces el archivo está vacío

`os.path.exists (arch-fisico)` # . Es Booleana. Retorna verdadero si el archivo existe

`os.rename ('alAlumno' , 'nuevoAlumno')` # renombra un archivo

`os.remove (os.getcwd()+/alumnos.dat)` # borrado del archivo físico

`print (os.listdir('ayed'))` # imprime todas los archivos del directorio actual

import os.path # permite identificar rutas locales / path = ruta

import io # para utilizar acceso directo - SEEK para parar el puntero

Cuál es el tamaño del archivo? ...en bytes!

Utilizaremos la función **getsize()** incluida en el módulo `os.path`

Notar que no es necesario que el archivo esté abierto con `open()` para usar esta función.
coincide con la posición del primer byte fuera del archivo

Ejemplo:

Tam= `os.path.getsize(ArcFisiAlu)`

0	123	Juan	...	
48	764	Pedro	...	
96	987	Luis	...	
144	109	Zoe	...	
192	985	Alan	...	
240	311	Ana	...	
287				

ArcFisiAlu

En este caso Tam= 288

Utilidad: recorrer el archivo hasta que el puntero coincida con el tamaño total de elementos que tenga el archivo..

Quién me dice dónde está el puntero?

Por lo general el puntero es gestionado automáticamente por las funciones básicas para gestionar archivos, por ejemplo OPEN(), DUMP() y LOAD()

El método **tell()** retorna el valor del puntero en forma de un número entero, es decir, devuelve el número del registro actual donde se encuentra en este momento.

Variable-lógica- tipo- Archivo. tell()

pos = ArcLogAlu.tell()

123	Juan	...	
764	Pedro	...	
987	Luis	...	
109	Zoe	...	
985	Alan	...	
311	Ana	...	



144

Notar que como
retorna un valor
entonces se lo puedo
asignar a una variable
y
No lleva parámetros.

en este caso
pos = 144
que es el 4to
guardado!!!

Cómo paro el puntero en un lugar determinado? Acceso directo

En algunas ocasiones el programador necesita posicionar el puntero en forma manual para acceder a ciertos datos del archivo.

El método **SEEK()** permite situarnos en el registro que deseamos cambiando el valor del puntero... *from-what 0-1-2*
Indica desde donde se hace el salto
offset
Indica cuantos bytes debe moverse

Variable-Logica-tipo-archivo.seek(*offset* , *from-what*)

io.SEEK_SET 0- saltar desde el principio del archivo

io.SEEK_CUR 1- saltar desde donde está el puntero en ese momento(que no es desde el principio ni desde el final)

io.SEEK_END 2- saltar desde el final del archivo

CUIDADO!!!

No es una función! No me devuelve NADA!!!

**Solo posiciona el puntero!
Tampoco lo trae!!**

Si no se indica el segundo parámetro al invocar a **SEEK()** se asume por defecto que salta desde el principio, es decir que su valor es 0

Ejemplos de seek:

- `arLogico.seek(0,0)` # me paro al comienzo del archivo, es decir, salto 0 bytes desde el inicio
- `arLogico.seek(0,2)` # me paro al final del archivo, es decir, salto 0 bytes desde el final
- `arLogico.seek(48,0)` # avanza 48 bytes desde el inicio
- `arLogico.seek(48,1)` # avanza 48 bytes desde donde está el puntero ahora
- `arLogico.seek(-48,2)` #retrocede 48 bytes desde el final

Cuántos (cantidad) registros hay grabados en el archivo?

```
ArcLogAlu.seek(0,0)
```

```
aux = pickle.load(arcLogAlu)    # lee y avanza el  
                                # puntero al byte siguiente  
TamReg = ArcLogAlu.tell()       #48
```

```
TamArchivo = os.path.getsize(ArcFisiAlu)    # 288
```

```
CantReg = int(TamArchivo / TamReg)    # 288/48=6 registros
```

Podemos usar int para que el
resultado sea entero o //

TamArchivo // TamReg

Cómo paro el puntero en un lugar determinado?

Ejemplo: sin importar el valor donde está ahora el puntero quiero moverlo al registro nro 2

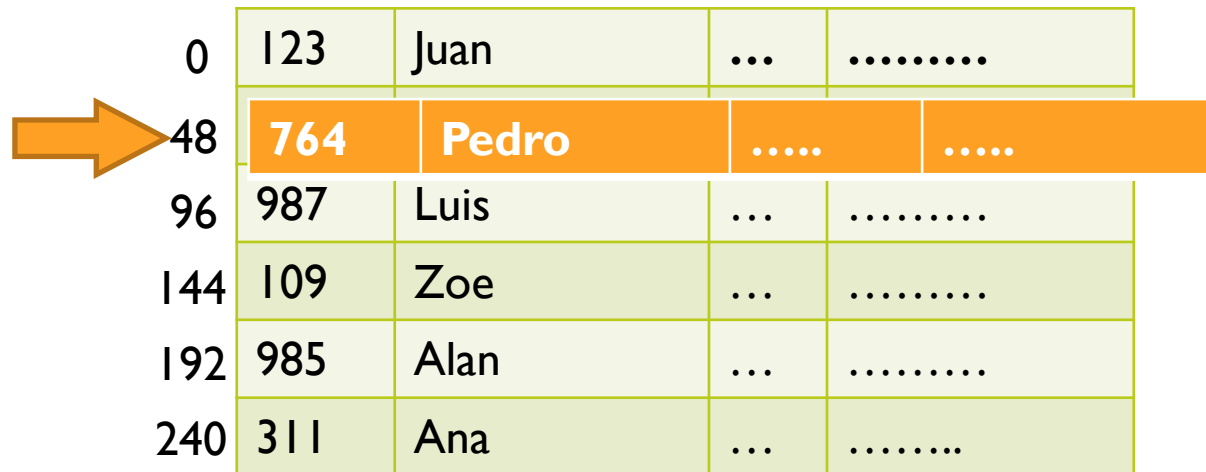
offset

Indica cuantos bytes debe moverse

from-what

0 Indica desde el principio

ArcLogAlu.seek(48, 0)



0	123	Juan	...	
48	764	Pedro		
96	987	Luis	...	
144	109	Zoe	...	
192	985	Alan	...	
240	311	Ana	...	

Luego de parar el puntero, para traerlo debo leerlo...

regAlu = pickle.load(ArcLogAlu)

Cómo paro el puntero al final del archivo sin importar dónde se encuentre..?

por ejemplo para cargar nuevos registros ...

offset
*Indica cuantos bytes
debe moverse
0 no mueve ninguno*

from-what
2 *Indica
desde el final*

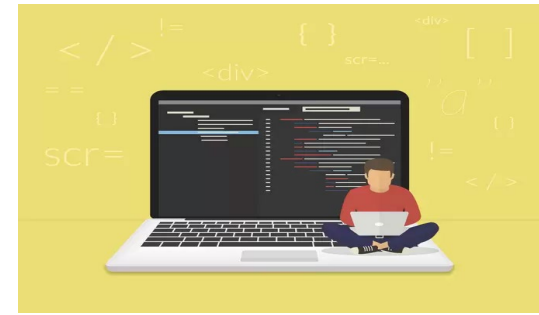
ArcLogAlu.seek(0,2)

0	123	Juan	...	
48	764	Pedro	...	
96	987	Luis	...	
144	109	Zoe	...	
192	985	Alan	...	
240	311	Ana	...	

→ 288

el puntero estará ubicado al final del archivo. Es decir, su valor sería el número del primer byte que está afuera del archivo.

Manos a la obra!!!....



Supongamos que seguimos con el ejemplo del archivo **Alumnos**, veamos cómo desarrollar el siguiente menú de opciones:

('MENU DE OPCIONES ')

- ('1- Nuevas Altas');
- ('2- Consultar el registro de un alumno determinado')
- ('3- Modifica el campo comisión de un alumno')
- ('4- Listado de promedios mayor a 8')
- ('5- Baja Lógica colocando ceros')
- ('0- Salir del programa')

('Ingrese su Opcion: ... ')

ExisteAbre Creo si no existe y/o abre

Pantalla

Leer (op)

$(op \geq 0) \text{ and } (op \leq 6)$

op

== 1

==2

== 3

== 4

== 5

== 0

altas

consulta

modifica

listado

bajas

$(op == 0)$

Cerrar

**También podríamos
cerrar dentro de la
opción salir del menú**

ExisteAbre

ArcFisiAlu = "c:\\ayed\\alumnos.dat"

Not os.path.exists(**ArcFisiAlu**)

v

f

ArcLogAlu = open (**ArcFisiAlu**, "w+b")

ArcLogAlu = open (**ArcFisiAlu**, "r+b")

Cerrar

ArcLogAlu.close()

Altas

ArcLogAlu.seek(0,2) Posiciono el puntero al final del archivo

regAlu=alumno()

leer (leg) Ingreso una variable simple que uso de fin de datos
validar

while leg != 0

Acum=0

leer (regAlu.nombre)

leer (regAlu.comision)

leer (regAlu.carrera) Se ingresan los campos del registro

For i in range (3):

leer (regAlu.notas[i])

Acum= Acum +regAlu.notas[i]

regAlu.promedio= Acum/3

regAlu.legajo= leg

FormateoRegistro()

pickle.dump(RegAlu, ArcLogAlu) Se graba el registro completo,
ArcLogAlu.flush() avanza el puntero y se lo empuja con el
flush

leer (leg)

**Deberíamos
controlar no se
ingrese dos veces el
mismo legajo...
debemos Buscar!**

**Mas adelante veremos
cómo Formatear el
largo del registro para
que sean todos de la
misma longitud**

Calcula el promedio

Sentencias de asignación

Altas mejorado

ArcLogAlu.seek(0,2) Posiciono el puntero al final del archivo

regAlu=alumno()

leer (leg)

while leg != 0

La búsqueda será secuencial o dicotómica dependiendo si está ordenado o no.

Pos = buscar(leg)

v

Pos == -1

f

Acum=0

Mostrar
(" ya existe")

leer (regAlu.nombre, regAlu.comisión, regAlu.carrera)

For i in range (3):

leer (regAlu.notas[i])

Acum= Acum +regAlu.notas[i]

regAlu.promedio= Acum/3

regAlu.legajo= leg

FormateoRegistro()

pickle.dump(RegAlu, ArcLogAlu)

ArcLogAlu.flush()

leer (leg)

Atención!
Tener en cuenta que si estamos buscando en forma dicotómica y no lo encuentra, el puntero no quedó al final, por lo tanto es conveniente posicionarlo seek(0,2) antes de formatear

Búsqueda Secuencial

def BuscaSec(leg):

Tam= os.path.getsize(ArcFisiAlu)	
pos=0	
ArcLogAlu.seek(0,0) Posiciono el puntero arriba	
RegAlu = pickle.load(arcLogAlu) Traigo el registro 0	
Mientras (ArcLogAlu.tell() < Tam) and (RegAlu.legajo != leg)	
pos= ArcLogAlu.tell() RegAlu = pickle.load(arcLogAlu)	
(RegAlu.legajo == leg)	
v	f
Return pos	Return -1 # el legajo ingresado no existe

Consulta de un registro

Leer (leg)	
Posi= BuscaSec(leg)	
Posi != -1	
v	f
ArcLogAlu.seek(Posi,0)	Mostrar ('el legajo ingresado no existe')
RegAlu = pickle.load(ArcLogAlu)	
Mostrar (RegAlu.legajo) Mostrar (RegAlu.nombre) Mostrar (RegAlu.carrera)	

No se graba nada porque es sólo un listado!

Modificación ó Actualización de un campo

Leer (leg)	
Posi= BuscaSec(leg)	
Posi!= -1	
v	f
RegAlu= Alumno()	Mostrar ('el legajo ingresado no existe')
ArcLogAlu.seek(Posi,0)	
RegAlu = pickle.load(arcLogAlu)	
Mostrar	
Ingresar- nuevos-datos	Acá también deberemos Formatear el largo del registro antes de grabar!!
ArcLogAlu.seek(Posi,0)	
FormateoRegistro()	
pickle.dump(RegAlu, arcLogAlu)	
arcLogAlu.flush()	CUIDADO!!!! Debemos volver a posicionar el puntero porque el load lo avanzó
Mostrar	

Ingresar dato a buscar
ENCONTRAR
MODIFICAR
RETROCEDER
GRABAR

**Acá también
deberemos
Formatear el largo
del registro antes
de grabar!!**

CUIDADO!!!!
**Debemos volver a
posicionar el
puntero porque el
load
lo avanzó**

Listado

Tam= os.path.getsize(ArcFisiAlu)		
v	Tam==0	
Mostrar ("No hay Alumnos registrados")	f	
	ArcLogAlu.seek(0,0)	
	while ArcLogAlu.tell() < Tam	
	RegAlu = pickle.load(arcLogAlu)	
	Mostrar	

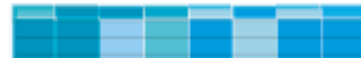
Este LOAD trae el registro a memoria y avanza el puntero

No se graba nada porque es sólo un listado!



UTN

Universidad Tecnológica Nacional
Ingeniería en Sistemas de Información



ARCHIVOS

Parte 2

Ing. María Inés Seguenzia

**Formateo de Registros
Ordenamiento
Búsqueda Dicotómica
y baja lógica
sobre Archivos**

Formatear registros

```
RegAlu.legajo = str(RegAlu.legajo)
```

```
RegAlu.legajo = RegAlu.legajo.ljust(4, ' ')
```

```
RegAlu.nombre = RegAlu.nombre.ljust(25, ' ')
```

```
RegAlu.comision = str(RegAlu.comision)
```

```
RegAlu.comision = RegAlu.comision.ljust(2, ' ')
```

```
For i in range (0,2)
```

```
    RegAlu.notas [i]= str(RegAlu.notas[i])
```

```
    RegAlu.notas [i] = RegAlu.notas[i].ljust(2, ' ')
```

```
RegAlu.prome = round(RegAlu.prome,2)
```

Los campos de un char como por ejemplo carrera 'S', 'M' ... no es necesario formatearlos.

redondeo el decimal y guarda solo 2 decimales después de la coma

Justifica a la izquierda left

.ljust(x, ' ')

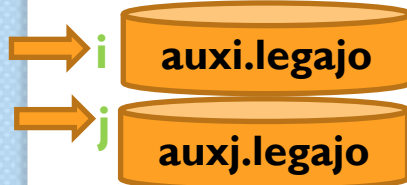
Con qué completará los que faltan hasta llegar a X

X Cantidad de caracteres que tendrá como máximo la cadena

Ordenamiento de archivo

Auxi, auxij = Alumno()

Variables
de tipo
registro



**Notar que no
necesitamos
formatear los
registros
porque los
estamos
cambiando de
lugar pero su
formato no
cambia!**

```
ArcLogAlu.seek(0,0)
```

```
auxi = pickle.load(arcLogAlu)
```

```
TamReg = ArcLogAlu.tell()
```

```
TamArch = os.path.getsize(ArcFisiAlu)
```

```
cantReg = int(TamArch / TamReg)
```

```
for i in range(0, cantReg-1):
```

```
    for j in range (i+1, cantReg):
```

```
        ArcLogAlu.seek (i*TamReg, 0)
```

```
        auxi = pickle.load(arcLogAlu)
```

```
        ArcLogAlu.seek(j*TamReg, 0)
```

```
        auxj = pickle.load(arcLogAlu)
```

```
        v auxi.legajo > auxj.legajo f
```

```
        ArcLogAlu.seek (i*TamReg, 0)
```

```
        pickle.dump(auxj, ArcLogAlu)
```

```
        ArcLogAlu.seek(j*TamReg, 0)
```

```
        pickle.dump(auxi, ArcLogAlu)
```

```
        ArcLogAlu.flush()
```

penúltimo
registro

último
registro

Búsqueda Dicotómica usando una Función

Def BUSCADICO(Leg:entero) : entera; entera sobre Archivos

ArcLogAlu.seek(0,0)		
RegAlu = pickle.load(arcLogAlu)		
TamReg = ArcLogAlu.tell()		
cantReg = int(os.path.getsize(ArcFisiAlu)) / TamReg		
inferior = 0	superior = cantReg-1	medio = (inferior + superior) // 2
ArcLogAlu.seek(medio*TamReg, 0)		
RegAlu = pickle.load(arcLogAlu)		
Mientras (inferior < superior) and int(RegAlu.legajo) != Leg		
v	Leg < int(RegAlu.legajo)	f
superior = medio - 1		inferior = medio + 1
medio = (inferior + superior) // 2		
ArcLogAlu.seek(medio*TamReg, 0)		
RegAlu = pickle.load(arcLogAlu)		
v	int(RegAlu.legajo) == Leg	f
return medio*TamReg		return -1

Baja lógica

Leer (leg)	
pos= BuscaSec(leg)	
Pos != -1 v	Lo ideal sería tener un campo especial para marcar si el registro está activo o no.
ArcLogAlu.seek(pos,0)	
RegAlu = pickle.load(arcLogAlu)	Mostrar ('el legajo ingresado no existe')
Mostrar	
Modificar algún campo	
ArcLogAlu.seek(pos,0) FormateoRegistro() pickle.dump(RegAlu, arcLogAlu) arcLogAlu.flush()	Preguntar si está seguro antes de confirmar la baja
Mostrar	